WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

# Harmlessness file

# WAC (Windows Artefacts Collector)

## Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

# WAC — Windows Artefacts Collector

Harmlessness file

| | |
|---|---|
| Software version | 4.0.0 |
| Document edition | October 2026 |
| Publisher | Aerospace Cyber Defence Centre of Excellence (CEC) |
| | French Air and Space Force Academy, Air Base 701, Salon-de-Provence |

This document establishes what WAC does to the machine it examines: every operation, the trace it leaves, and what it never does. It gives the evidence for these statements — the table of the functions `WAC.exe` can call, the checks that refuse a faulty build, the measurement made on a test machine — and the procedure that lets anyone redo them.

# Contents


**1 Purpose** **2**

1.1 What “harmless” means here . . . 2

1.2 What this document does not cover . . . 2

**2 Inventory of the traces** **3**

2.1 The intervention . . . 3

2.2 The run of WAC.exe by Windows . . . 3

2.3 Measured against KAPE and the EZ tools . . . 4

2.4 WAC’s operations . . . 5

2.5 What WAC never does . . . 7

**3 The evidence** **9**

3.1 The import table of WAC.exe . . . 9

3.2 The guards of the build . . . 10

3.3 The investigation log . . . 10

**4 Validation procedure** **11**

4.1 Step 1: the delivered binary is the one of the sources . . . 11

4.2 Step 2: the import table . . . 11

4.3 Step 3: the measurement on a test machine . . . 11

4.4 Step 4: the log of a real collection . . . 12

**5 Limits** **13**

CHAPTER 1

# Purpose

## 1.1 What “harmless” means here

A collection is made on a machine that is also evidence. A tool that changes it destroys part of what it came for, and mixes its own traces with those of the activity under study. WAC is said to be *harmless* in the following sense:

1. it **writes nothing** to the volumes of the examined machine — no file, no registry, no event in a log; what a monitoring in place (Sysmon, auditing, EDR) records of its execution is described in chapter 2;
2. it **changes no metadata** of the files it reads: no access date, no change number in the NTFS journal;
3. it **sends nothing over the network** and resolves no name;
4. it **installs nothing and does not persist**: no service, no driver, no task, no start-up key; once it has exited, all that remains of it are the traces Windows produces for any program run;
5. it **does not alter the running machine**: no process opened, no service stopped or reconfigured, no privilege changed;
6. what it cannot avoid is **listed**, here and, collection by collection, in `investigation.json`.

## 1.2 What this document does not cover

- The correctness of the data collected: that is the subject of the *Validation report*.
- The day-to-day use of the tool: that is the subject of the *User manual*.
- The update of the trust lists (`--update-trust`): it runs on the analysis workstation, never on the examined machine, and it is the only mode that opens a network connection.

CHAPTER 2

# Inventory of the traces

The traces fall into three families: those of the intervention itself (plugging in a medium, opening a session), those Windows produces for any program run, and those of WAC's own operations. The first two exist whatever the tool; they are listed so that the analyst recognises them rather than mistaking them for the activity under study.

## 2.1 The intervention

| Action | Trace |
|---|---|
| Plugging in the collection medium | Keys `USBSTOR`, `USB`, `MountedDevices`, `Windows Portable Devices`, `VolumeInfoCache`; file `setupapi.dev.log`; logs `Microsoft-Windows-Partition/Diagnostic` (1006), `Kernel-PnP`, `DeviceSetupManager`. |
| Opening a session | Security events 4624 and 4672 (special privileges); for an account never logged on to the machine, creation of its profile. |
| Elevating the command prompt | Run of `consent.exe` (User Account Control) and its own execution traces. |
| Starting `cmd.exe` | Execution traces of `cmd.exe` (next section). |

## 2.2 The run of WAC.exe by Windows

Windows produces them for any program; WAC cannot prevent them and does not try to erase them — erasing would be writing to the machine.

| Artefact | Trace |
|---|---|
| Prefetch | `C:\Windows\Prefetch\WAC.EXE-<hash>.pf`, written by Windows a few seconds after the start, then at the end — for a `WAC.exe` started from the machine's disk: started from a USB key, measured under Sysmon, it got none. |

| Artefact | Trace |
|---|---|
| Amcache, ShimCache | One entry for `WAC.exe` (path, size, date, SHA-1 fingerprint for Amcache). Measured: the Amcache entry is written during the collection, the ShimCache one at the machine's shutdown, as `SIGN.MEDIA=<signature> WAC\WAC.exe` for a key. |
| BAM | One value for `WAC.exe` under the user's key, dated from the end of the run — again when started from the machine's disk: none from the USB key. |
| SRUM | Resource usage of the process, aggregated by Windows. |
| Logs | Security 4688 and 4689 if process creation auditing is on; Sysmon 1 if Sysmon is installed; telemetry of the antivirus or EDR (scan of `WAC.exe` and, for an EDR, a possible alert on the raw opening of the volume). |
| Memory | WAC's pages take up memory during the collection and may push others out to the page file. |
| RADAR | If `WAC.exe`'s memory crosses Windows' threshold, its leak detector records it: a value under `HKLM\SOFTWARE\Microsoft\RADAR\HeapLeakDetection\DiagnosedApplications\WAC.exe` (`LastDetectionTime`) and event 1014 of `Microsoft-Windows-Resource-Exhaustion-Resolver/Operational`. Seen twice on the test machine, in September and October 2026, never during the cycles of version 3.0.0. |

> **Keep in mind**
>
> WAC collects these very artefacts: it therefore shows up in its own results (Prefetch of the next collection, BAM, Amcache). This is expected, and dated: the `Collection.StartUtc` — `EndUtc` interval of `investigation.json` sets them apart.

## 2.3 Measured against KAPE and the EZ tools

On a clone of the test machine, frozen with Sysmon, each tool is started from a USB key from the same state; C:'s change journal, bounded before and after the tool, and Sysmon say what it wrote, and who wrote it; after the shutdown, the disk's hives are read again offline and set against those of the frozen state, key by key and value by value (comparison file, chapter "Traces left on the examined machine"). Measured on 29 and 30 October 2026, on version 3.0.0 and the fix of the module list described below.

| Tool | What its processes write on the machine |
|---|---|
| WAC | Nothing: no file, no registry value. |
| KAPE (targets) | 4 `ZoneMap` values of Internet Settings in the user's hive. |
| KAPE and its modules | The same values, rewritten by 14 processes; 8 RAS tracing keys (`HKLM\SOFTWARE\Microsoft\Tracing`) created by 4 tools; 28 files on C: (.NET runtime breadcrumbs, sentinel files in the user's Temp). |
| EZ tools live | No value and no file, but creation opens of policy and network keys. |

The hives read again offline confirm it: what changes in them during WAC's collection, beyond what the control run changes too, is the Amcache entry of `WAC.exe` and Windows' activity in the measuring window (WaaSMedic, Defender, scheduled tasks, WMI, Windows Update, OneDrive), which Sysmon gives other processes. No tool had an event written in Windows' own logs. But all are recorded by the monitoring installed, as the previous section foresees: Sysmon noted, for WAC, its process, its 3 raw reads (C:, the EFI partition, the physical disk), the 25,532 files it created, all on the key, and the processes it opened — with limited query through Terminal Services, then with query and memory read for their module list; process creation auditing, off on the clone, would have recorded a 4688 and a 4689. For all, Windows adds an Amcache entry during the run and a ShimCache entry at shutdown; all read the volumes raw. KAPE and the EZ tools also need the .NET 9 runtime on the examined machine.

## 2.4 WAC's operations

Every operation is recorded in `investigation.json` with, in its `Footprint` field, the trace it is expected to leave. The table below lists them all.

| Operation | Trace |
|---|---|
| Raw reading of the volume (`\\.\C:`) | One handle opened **read-only**, uncached (`FILE_FLAG_NO_BUFFERING`), on each volume actually read. No file is opened through the system: no access date changed, no USN journal entry, no lock. If object-access auditing is set on the volume: Security 4656/4663. Uncached, the reading does not replace the content of the machine's file cache either. |

| Operation | Trace |
|---|---|
| Registry hives | Copied by the raw reading, then read on the copy. No key of the live registry is opened: `WAC.exe` imports no `Reg*` function. The transaction logs are replayed on the copy, with an undo journal. In the SECURITY hive, the LSA secret names and dates (never their value), the cached-logon counts (never their content) and the account rights are read. In the SAM hive, the password hashes are decrypted and the reset security questions read in the clear: a deliberate collection, so that WAE can flag weak passwords. All of it on the copy, read-only; the examined machine is not touched, and a hash is one-way. |
| Event logs (`--events`) | Read by the raw reading: the event log service is not called upon, and this reading produces no event. |
| FAT and exFAT volumes | Read raw through the volume device (`\\.\X:`), read only, fixed and removable volumes alike: boot sector, FAT and directories, never a file opened, no access date updated. The volume holding the collection is not read. |
| Boot records (`--boot_records`) | Read raw through the device of each physical disk (`\\.\PhysicalDriveN`), opened read only: the first mebibyte, each partition's first sectors, the EFI system partition's files through WAC's FAT reader — the partition is not mounted, no letter is given to it. The collection medium's disk is not read, unless it holds the system. |
| Entra ID token caches (`--cloud_identity`) | Read by the raw reading and collected as they are, encrypted by DPAPI: WAC neither decrypts nor uses them; no cloud service is contacted. |
| Directory and policies (`--active_directory`, `--group_policies`) | Read by the raw reading: `ntds.dit`, its logs and `SYSVOL`. No LDAP query, no replication: `WAC.exe` imports no LDAP library, so the controller records no directory access on WAC's behalf. The logs are replayed on the working copy. |
| Web servers (`--web_server`) | Configuration, sites, logs and databases read by the raw reading. Neither the web server nor any database engine is queried, stopped or restarted; a live database is taken as the disk holds it. |
| Binaries (`--binary`) | Read by the raw reading; signatures and catalogues verified in memory. No certificate store opened, no online revocation check. |
| Processes | One kernel snapshot (`CreateToolhelp32Snapshot`, the process list only) and one query to the Terminal Services service (`WTSEnumerateProcessesW`) for the owners, which opens each process with limited query (`0x1000`), LSASS included. |

| Operation | Trace |
|---|---|
| Process modules | Each process opened with query and memory read only (`OpenProcess`, `PROCESS_QUERY_INFORMATION` and `PROCESS_VM_READ`, granted as `0x1410`), its list read through psapi: no write, thread or injection right. Those the kernel protects (LSASS under LSA protection, services, antivirus) refuse it, and the refusal is recorded. Up to version 3.0.0, the Toolhelp module snapshot opened each process itself with full access (`PROCESS_ALL_ACCESS`, measured under Sysmon): replaced. |
| Sessions | One read query to LSASS (`LsaEnumerateLogonSessions`, `LsaGetLogonSessionData`). |
| State of the services | One read-only enumeration of the service manager, rights `SC_MANAGER_ENUMERATE_SERVICE \| SC_MANAGER_CONNECT`. No service is opened; the configuration comes from the copied hive. |
| Network state (`artefacts.network`) | In-memory reading of the TCP/IP stack's tables through `iphlpapi.dll`, loaded at that moment only. No packet sent, no name resolved. |
| Clock and boot | In-memory queries to the kernel (`NtQuerySystemInformation`, `GetTickCount64`). |
| Machine settings | System drive, code page, time zone, volumes and serial numbers: in-memory queries, with no file opened. |
| Process token | Reading of WAC's own privileges, to check the administrator rights. No privilege is enabled or changed. |
| Writing the results | To the `--output` folder only, relative to the current directory — the collection medium. |

**Limit**

WAC writes to the `--output` folder of the current directory. `--output` (and `output:` in `wac.yml`) is a folder name only: no separator, no drive (`C:case`), not `..`; anything else is refused before any collection, so that it cannot point at the examined volume. Started from a folder of the examined disk, however, it would write its results there: files created, records of the `$MFT` and of the USN journal, blocks overwritten. **Start it from the collection medium**, which must be the current directory.

## 2.5 What WAC never does

- Create a shadow copy (VSS), call COM or WMI: none of these libraries is imported. The existing copies are read raw on the volume (catalog and stores of the differential area),

through the same read-only access as the rest: none is mounted, exposed or asked of the service.

- Recover a deleted file through a tool, or by writing to the volume: the content of the deleted files is read raw (records of the `$MFT`, `$Bitmap`, free clusters, directory index blocks, and at level 3 the whole free space; FAT and exFAT volumes through their device), through the same read-only access as the rest, and copied to the collection medium only. A cluster in use is not read.
- Open a key of the live registry, or flush it to the disk (`RegFlushKey`).
- Load a driver, create or change a service, a scheduled task, a start-up key.
- Open a process, read its memory, inject code.
- Start a process: no function that creates one is imported (`system`, `CreateProcess`, `ShellExecute`, `WinExec`). Clearing the screen at start-up used to run `cmd.exe /c cls`, with its execution traces; the compared measure of the traces left showed it, and the screen is now cleared through the console API.
- Enable a privilege (`AdjustTokenPrivileges`): the raw reading of a volume needs administrator rights only.
- Open a network connection, resolve an account name with the domain controller (`LookupAccountSid`): accounts are named offline, from the copied hives.
- Write to a temporary file of the system.

CHAPTER 3

# The evidence

## 3.1 The import table of WAC.exe

A Windows program can only call a system function if it imports it, or if it loads its library dynamically. `WAC.exe`'s import table is therefore the exhaustive list of what it can ask of the system — apart from the two dynamic loads described below. It is read without running the program, on Linux:

```
$ x86_64-w64-mingw32-objdump -p WAC.exe | grep -E 'DLL Name|^\s+[0-9a-f]+\s+[0-9]+ '
```

Apart from `msvcrt.dll` (the C library):

**ADVAPI32.dll**
: `OpenSCManagerW`, `EnumServicesStatusExW`, `CloseServiceHandle` (state of the services); `OpenProcessToken`, `GetTokenInformation`, `GetUserNameW`, `GetLengthSid`, `IsValidSid` (WAC's own token); `AllocateAndInitializeSid`, `CheckTokenMembership`, `FreeSid` (membership of the administrators group, the only elevation Windows XP knows). No `Reg*` function, no function that changes a service.

**Secur32.dll**
: `LsaEnumerateLogonSessions`, `LsaGetLogonSessionData`, `LsaFreeReturnBuffer`.

**WTSAPI32.dll**
: `WTSEnumerateProcessesW`, `WTSFreeMemory`.

**PSAPI.DLL**
: `EnumProcessModules`, `GetModuleFileNameExW`: the module list of each process, opened by `OpenProcess` with query and memory read.

**bcrypt.dll**
: `BCryptOpenAlgorithmProvider`, `BCryptSetProperty`, `BCryptGetProperty`, `BCryptGenerateSymmetricKey`, `BCryptDecrypt`, `BCryptDestroyKey`, `BCryptCloseAlgorithmProvider`: the AES decryption of the SAM's hashes, at the conversion only. Computation in memory, no file, no registry.

**KERNEL32.dll**
: Files, memory, threads, time, console, volumes, process snapshot and opening. The functions that write or delete (`DeleteFileW`, `MoveFileExW`, `RemoveDirectoryW`, `CreateHardLinkW`, `SetEndOfFile`) serve the output folder, or belong to the statically linked C++ library (`std::filesystem`) without WAC calling them elsewhere: every deletion or rename in the source code is on the output folder, its working copy or its staging folder.

**Dynamic loads.** Three libraries only, from `System32` (`LOAD_LIBRARY_SEARCH_SYSTEM32` or, under a Windows that does not know that flag, their full path in that folder): `iphlpapi.dll`, for the

network state, when `artefacts.network` is asked for; `winhttp.dll`, for `--update-trust` alone, and `esent.dll`, which replays a copied ESE database's logs (WebCache, SRUM, BITS) into the working copy, at conversion alone — neither of the last two runs on the examined machine: WAC collects there, and never converts there.

**`WAC32.exe`**, the 32-bit WAC (Windows XP SP1 to Windows 8 x86), is built from the same sources and imports the same functions, except those XP lacks: the fourteen functions that came with Vista (the whole of `bcrypt.dll` among them) are defined by WAC itself, which calls Windows' function where it exists and replaces it otherwise with what XP offers. `bcrypt.dll` is then loaded from `System32`, at conversion alone. Added are `CreateSemaphoreW`, `ReleaseSemaphore` (the threads of the 32-bit C++ library) and `LoadLibraryA`, which its start-up code calls only for a runtime library already present in the process. The build confronts every import of `WAC32.exe` with the exports of Windows XP SP1.

## 3.2 The guards of the build

`build-windows.sh` refuses to produce a `WAC.exe` that would break these rules. After the link, it inspects the import table and fails:

- if `WAC.exe` imports a function that resolves names by querying the domain (`LookupAccountSid`, `LookupAccountName`, `LsaLookupSids`, `LsaLookupNames`);
- if it imports a network library statically (`winhttp`, `wininet`, `ws2_32`, `wsock32`, `iphlpapi`);
- if it depends on a MinGW runtime library: the executable must be self-contained, so as to drop nothing beside it.

Before even building, it refuses a compiler whose *thread model* is not "win32". A gcc with "posix" threads — MSYS2's on Windows — links winpthreads into `WAC.exe`, and with it 38 imports WAC never calls: `SetSystemTime`, `SetThreadContext`, `SuspendThread`, `TerminateProcess`, `SetProcessAffinityMask`… They change nothing in what WAC does, but they widen what it *could* ask of the system, and they are precisely the names a defence tool or an analyst notices on the examined machine. Measured: with Debian's gcc (Linux) or MinGW-Builds' (Windows), both with "win32" threads and on msvcrt, those imports are absent, and the two tables differ only by eight input/output functions of the C library. "win32" names the thread model, not an architecture: `WAC.exe` is x86_64 either way.

The build is reproducible: the same sources give the same `WAC.exe`, byte for byte, and `SHA256SUMS` attests it.

## 3.3 The investigation log

Every collection writes, in `investigation.json`, the ordered list of its operations, each with its target, its result and its `Footprint` field. A gap between what this file says and what the log of a real collection says therefore shows collection by collection, and not only during a test campaign.

CHAPTER 4

# Validation procedure

This procedure is to be redone with every version, and with every change that touches a system call. The first two steps are done on the analysis workstation, with no Windows machine.

## 4.1 Step 1: the delivered binary is the one of the sources

```
$ ./build-windows.sh
$ sha256sum -c bin/SHA256SUMS
```

Expected: the build ends with no error, no warning, and shows the three checks: “Static C++ runtime”, “No network name resolution imported”, “No network library imported”. The `WAC.exe` built and the one in `bin` have the same fingerprint.

## 4.2 Step 2: the import table

```
$ x86_64-w64-mingw32-objdump -p bin/WAC.exe | grep -E 'DLL Name|^\s+[0-9a-f]+\s+[0-9]+ '
```

Expected: the five libraries and the functions of section 3, and no other. A new function is explained in this file before the release.

## 4.3 Step 3: the measurement on a test machine

The automatic cycle (`vmtest/run-wac-test.sh`, see the *Testing guide*) runs WAC in a virtual machine (Windows 11, and Vista to 8.1) and checks, on witness files that WAC reads, that none of their metadata changes:

1. before the collection, the cycle creates witness files that WAC is bound to read (a macro document and a script, which `--binary` collects) and records, through PowerShell, their four dates and the number of their last change in the NTFS journal (`fsutil usn readdata`);
2. it runs the collection;
3. it records the same values again: they must be identical;
4. to prove that the check can fail, it then reads a witness through the API (`Get-Content`) and checks that its access date, for its part, changes when Windows updates it.

The result is in the cycle's report (`harmlessness.txt`). On 29 October 2026, on the code of version 3.0.0: witnesses unchanged under Windows 11 and under Windows 8.1. Under Windows 11, the read through Windows changed the access date: the check can fail. Windows 8.1 kept no access dates on the volume: there, the check proves the USN only. On 4 November 2026, on the code of 4.0, on a domain controller under Server 2022, with access dates turned on for the measure (`fsutil behavior set disablelastaccess 0`, restored afterwards): witnesses unchanged, while one read through Windows changed the access date. The check can fail there too. On this controller as Windows leaves it, access dates are off and the check proves the USN only. On 7 and 8 November 2026, on the code of 4.0.0, WAC started from a USB key as in the field: witnesses unchanged in every full cycle, under Windows 11 and under each server from Server 2008 R2 to 2025, Core and roles included; under Windows 11, one read through Windows again changed the access date. On 12 and 13 November 2026, on the code of 5.0.0 (applications' traces included, read raw like the rest): witnesses unchanged in every full cycle, under Windows 11, Server 2025 (domain controller included), the 2022 roles and Server 2008 R2; under Windows 11, one read through Windows again changed the access date, on the servers access dates are off and the check proves the USN number only. The measurement under Sysmon, against KAPE, is still the one on the Windows 11 workstation: it was not repeated on a server.

## 4.4 Step 4: the log of a real collection

On a real collection, read `investigation.json` again: every operation must belong to the table of chapter 2, with the matching `Footprint`. An operation this file does not know is a defect to report.

CHAPTER 5

# Limits

**Output folder**

`--output` cannot leave the current directory, but WAC does not check that the current directory is on a volume other than those it examines: it is up to the operator to start it from the collection medium.

**Monitoring**

An EDR or an audit policy may log the raw opening of the volume, the enumeration of processes and sessions, and the opening of each process with memory read for its module list (Sysmon, event 10; LSASS, when not protected, among them). These logs are written by the machine, not by WAC; they are expected.

**Memory**

WAC does not capture memory, but it occupies some: a page pushed out to the page file during the collection is not a trace on the disk WAC reads, but it changes the state of the memory a tool started afterwards would capture. Capture memory, if it is needed, **before** WAC.

**Measurement**

Step 3 is about witness files of a virtual machine: it establishes how the reading behaves, not that Windows itself writes nothing during the collection.